Queremos desarrollar una aplicación para **gestionar empleados**.
Los requisitos son:

1. **Registrar empleados** con nombre y salario. Puede haber distintos tipos de empleados (Fijos, Temporales, Colaboradores,...)
2. **Calcular el salario anual** de cada empleado, que dependerá de su salario mensual y bonificaciones
3. Permitir **diferentes tipos de bonificaciones** según el tipo de empleado (Ej: empleados fijos y temporales).
4. Poder **generar informes** de empleados en diferentes formatos (ej: consola, CSV).
5. El sistema debe estar diseñado de forma que **sea fácil añadir nuevos tipos de empleados, bonificaciones o informes sin modificar el código existente**.

SOLUCIÓN:

Aplicaremos los siguientes principios:

- **SRP (Single Responsibility Principle)** → Cada clase con una única responsabilidad.

- **OCP (Open/Closed Principle)** → Poder extender sin modificar código existente (para añadir nuevos tipos de empleados o informes).

- **DIP (Dependency Inversion Principle)** → La lógica de alto nivel dependerá de interfaces, no de implementaciones concretas.

CÓDIGO:

```
// SRP: Clase que representa un empleado y solo gestiona información del empleado
abstract class Empleado {
    protected String nombre;
    protected double salarioMensual;

    public Empleado(String nombre, double salarioMensual) {
        this.nombre = nombre;
        this.salarioMensual = salarioMensual;
    }

    public String getNombre() { return nombre; }
    public double getSalarioMensual() { return salarioMensual; }

    // OCP: Método abstracto que permite calcular salario anual
```

```
    //según tipo de empleado
    public abstract double calcularSalarioAnual();
}
```

Tipos concretos de empleados:

```
// Implementación para empleado fijo
class EmpleadoFijo extends Empleado {
    public EmpleadoFijo(String nombre, double salarioMensual) {
        super(nombre, salarioMensual);
    }
    @Override
    public double calcularSalarioAnual() {
        return salarioMensual * 12 + 1000; // bonus fijo
    }
}

// Implementación para temporal
class EmpleadoTemporal extends Empleado {
    public EmpleadoTemporal(String nombre, double salarioMensual) {
        super(nombre, salarioMensual);
    }
    @Override
    public double calcularSalarioAnual() {
        return salarioMensual * 12; // sin bonus
    }
}
```

Interfaz para informes:

```
// DIP: Interfaz para generación de informes
interface Informe {
    void generar(List<Empleado> empleados);
}
```

Implementaciones:

```
// Implementación de informe en consola
class InformeConsola implements Informe {
    @Override
```

```
    public void generar(List<Empleado> empleados) {
        for (Empleado e : empleados) {
            System.out.println(e.getNombre() + " - " +
e.calcularSalarioAnual());
        }
    }
}
// Implementación de informe CSV
class InformeCSV implements Informe {
    @Override
    public void generar(List<Empleado> empleados) {
        System.out.println("nombre,salarioAnual");
        for (Empleado e : empleados) {
            System.out.println(e.getNombre() + "," +
e.calcularSalarioAnual());
        }
    }
}
```

Clase de lógica de negocio que utiliza el sistema:

```
// Clase de alto nivel (depende de abstracciones → DIP)
class Empresa {
    private List<Empleado> empleados = new ArrayList<>();

    public void agregarEmpleado(Empleado empleado) {
        empleados.add(empleado);
    }

    public void generarInforme(Informe informe) {
        informe.generar(empleados);
    }
}
```

Clase de prueba o capa de presentación:

```
// clase de prueba
```

```
public class Main {
    public static void main(String[] args) {
        Empresa empresa = new Empresa();
        empresa.agregarEmpleado(new EmpleadoFijo("Ana", 2000));
        empresa.agregarEmpleado(new EmpleadoTemporal("Luis", 1500));

        // OCP: podemos usar diferentes tipos de informes
        // sin modificar Empresa
        empresa.generarInforme(new InformeConsola());
        empresa.generarInforme(new InformeCSV());
    }
}
```